Hardware Based Handwritten Digit Recognition for MNIST Dataset

Hardik Raina
ML Engineer, Agnisys

ai@agnisys.com

December 18, 2024

Abstract- In recent years, hardware acceleration has become a significant area of research for speeding up machine learning tasks[11], particularly deep learning applications. Field-Programmable Gate Arrays (FPGAs) offer unique advantages in terms of parallelism, reconfigurability, and energy efficiency compared to traditional CPU and GPU architectures[12]. The handwritten digit recognition task, particularly using the MNIST dataset, is a classic problem in the machine learning community and serves as a benchmark for evaluating model performance. This paper focuses on the direct implementation of a handwritten digit recognition model on an FPGA to explore how hardware-based solutions can enhance the speed and efficiency[7] of deep learning models. The primary goal is to detail the FPGA configuration process, data precision handling, and overall performance analysis.

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

1 Introduction

1.1 Objective

The objective of this paper is to explore the potential of hardware acceleration in the domain of deep learning, specifically for the task of handwritten digit recognition. This research aims to detail the direct implementation of a handwritten digit recognition model[6] using an FPGA (Field-Programmable Gate Array) and demonstrate the advantages of deploying such solutions over traditional CPU and GPU architectures. The main focus will be on the FPGA configuration, data precision considerations, and the comparison of performance metrics to establish the feasibility and benefits of hardware-based deep learning solutions. Agnisys IDS tools will be employed throughout the design and verification phases, including FPGA configuration and implementation, to ensure efficient and accurate development of the hardware solution.

1.2 Background

With the rapid growth of deep learning[4], there has been an increasing demand for efficient computing resources to handle complex models and large datasets. Traditional architectures, such as CPUs and GPUs, provide high performance but often fall short in terms of energy efficiency and customization for specific tasks[9]. FPGAs, known for their reconfigurable hardware and parallel processing capabilities, present a compelling alternative. These programmable devices can be tailored to perform certain operations more efficiently, enabling significant speedups in computation while reducing power consumption[9]. The MNIST dataset, comprising 28x28 pixel grayscale images of handwritten digits, has long been a standard benchmark for evaluating the performance of machine learning algorithms[6]. Implementing a digit recognition model directly on FPGA allows for an exploration of the feasibility and benefits of accelerating deep learning tasks with custom hardware[7].

1.3 Scope

This paper focuses on the end-to-end process of implementing a handwritten digit recognition model on

an FPGA, covering the key aspects of the design and optimization steps necessary for such an implementation. It will delve into the FPGA configuration process, which includes hardware synthesis, data precision handling, and parallel processing techniques. Additionally, this work will address the optimization strategies employed to balance the trade-offs between model accuracy, speed, and resource utilization. The paper will also include a detailed performance analysis, comparing FPGA-based implementation with conventional CPU and GPU approaches to highlight the potential gains in speed and energy efficiency. By exploring these aspects, this research aims to provide insights into how hardware-based solutions can contribute to the future of efficient deep learning.

2 MNIST Dataset Overview and Preprocessing

2.1 Dataset Description

The MNIST dataset is one of the most widely used datasets in the machine learning community, serving as a benchmark for evaluating image classification models. It consists of 70,000 grayscale images of handwritten digits ranging from 0 to 9. Each image has a resolution of 28x28 pixels, and the dataset is split into 60,000 training images and 10,000 test images. The pixel values in each image range from 0 to 255, representing the grayscale intensity of the image. The simplicity and accessibility of MNIST make it an ideal dataset for testing new machine learning algorithms and hardware-based implementations.

2.2 Preprocessing

Preprocessing is a crucial step when preparing the MNIST dataset for training and inference[5]. For FPGA implementation, this step must be optimized to balance performance and resource usage[10]. Key preprocessing operations include:

- **Flattening:** A Flatten layer is used to convert the 2D 28x28 image into a 1D vector with 784 elements. This transformation simplifies data handling and is an essential step in adapting the image data for subsequent neural network layers. Implementing this Flatten operation directly in the FPGA logic allows the data to be efficiently prepared for further processing without needing additional software-based reshaping.
- **Scaling:** To ensure that the neural network processes input data in a normalized form, the pixel values are scaled from a range of 0–255 to 0–1. This scaling can be implemented as a simple division by 255 during the data input phase, which can be achieved using hardware-efficient arithmetic operations. Implementing this step in hardware helps minimize the processing overhead typically associated with data normalization in software-based models.

For FPGA implementation, these preprocessing steps can be parallelized and optimized to execute simultaneously with data reading, thus reducing the data throughput latency[10].

3 AI Model Overview

For the handwritten digit recognition task, a Convolutional Neural Network (CNN) is often used due to its ability to capture spatial hierarchies in image data and its relatively low computational complexity compared to deeper or more complex networks. A basic CNN architecture used for this paper includes the following layers[3]:

- **Input Layer:** Accepts the 784-element vector from the flattened image.
- **Convolutional Layers:** Several convolutional layers (e.g., with 3x3 or 5x5 kernels) are applied to extract features from the input data. These layers leverage filters to identify patterns such as edges, corners, and more complex shapes in the images.
- **Activation Function:** A non-linear activation function like ReLU (Rectified Linear Unit) is applied after each convolutional operation to introduce non-linearity and enable the network to learn complex functions.

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

- **Pooling Layers:** Max pooling layers are used to down-sample the feature maps and reduce the spatial dimensions while preserving the most important features.
- **Fully Connected Layers:** After convolutional and pooling layers, the flattened output is passed through one or more fully connected layers to perform classification based on extracted features.
- **Output Layer:** A softmax layer provides probabilities for each of the 10 classes (digits 0–9), outputting the final prediction.

For FPGA implementation, the CNN architecture needs to be optimized for parallel execution and hardware constraints. This involves optimizing layer computations for parallelism, using fixed-point or custom-precision arithmetic, and integrating efficient memory handling strategies to minimize latency.

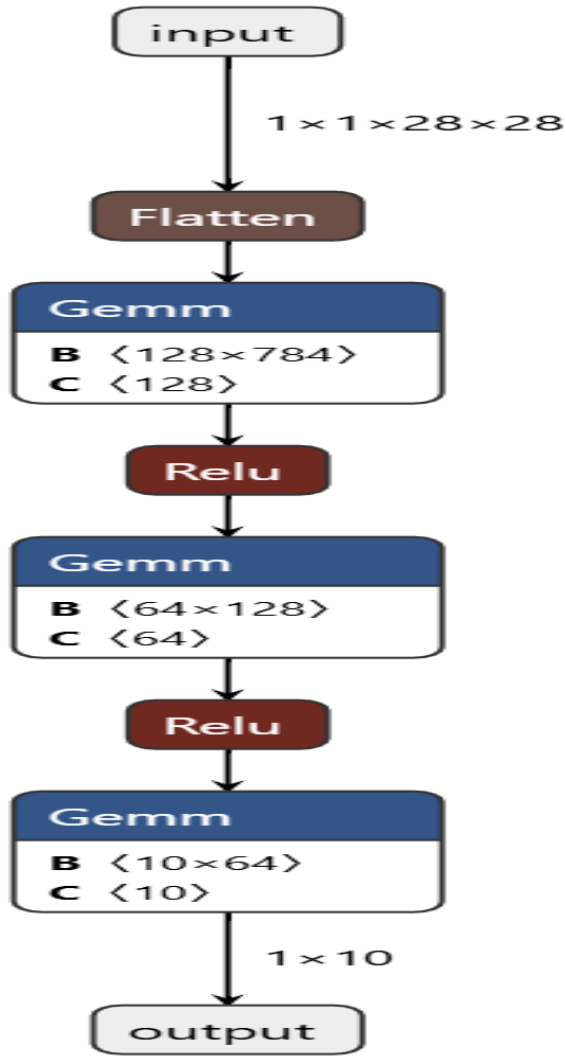

Figure 1: Exemplified Model Architecture

4 Functions Required

A bit of a foreward as we proceed to discuss the mathematical functions involved for our use case.

1. In standard implementations the value x is a ***fp32 (32 bit floating point)*** number.

4.1 ReLU

In the context of artificial neural networks, the rectifier or ReLU (rectified linear unit) activation function is an activation function defined as the non-negative part of its argument[8].

$$\mathrm{ReLU}(x) = \max(0, x)$$

4.2 Exponential

The exponential is a common mathematical expression. Used in the context of machine learning to increase the output values generated from inputs in *logits*. Considering the sheer complexity of implementing such a function via lookup tables, we can opt into using a mathematical approximation of the exponential function using **Taylor Expansion**[2] for the exponential function.

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

4.3 Softmax

The softmax function is a function that turns a vector of K real values into a vector of K real values that sum to 1. The output of the function is always between 0 and 1, which can be used as a probability score.

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

Here, the symbol ! denotes the factorial of an integer which can be stored in a lookup table.

$$1! = 1$$

$$2! = 2 * 1 = 2$$

$$3! = 3 * 2 * 1 = 6$$

$$4! = 4 * 3 * 2 * 1 = 24$$

$$\dots$$

4.4 GeMM

GeMM stands for ***General Matrix Multiplication*** and is a critical operation used for wide variety of deep learning tasks[1].

$$C = \alpha AB + \beta C$$

where:

- A is an $m \times k$ matrix,
- B is a $k \times n$ matrix,
- C is an $m \times n$ matrix,
- α and β are scalars that can be used to scale the matrices $A \cdot B$ and C respectively.

This operation can be expanded as:

$$C_{ij} = \alpha \sum_{k=1}^{K} A_{ik} B_{kj} + \beta C_{ij}$$

for $i = 1, \dots, M$ and $j = 1, \dots, N$.

Note that α and β are 1 for most standard implementations.

5 Model Training

5.1 Forward Pass

The forward pass of the neural network can be expressed mathematically as follows:

Given an input vector $x \in \mathbb{R}^{28\times28}$ (28x28 image flattened into a 1D vector):

1. **Flattening**:

$$\mathbf{x}_{\text{flat}} = \text{Flatten}(x) \quad \text{where} \quad \mathbf{x}_{\text{flat}} in \mathbb{R}^{784}$$

2. **First Fully Connected Layer (fc1)**:

$$\mathbf{z_1} = \mathbf{W_1}\mathbf{x}_{\text{flat}} + \mathbf{b_1} \quad \text{where} \quad \mathbf{W_1} \in \mathbb{R}^{128\times784}, \quad \mathbf{b_1} \in \mathbb{R}^{128}$$

3. **ReLU Activation (after fc1)**:

$$\mathbf{a_1} = \text{ReLU}(\mathbf{z_1}) = \max(0, \mathbf{z_1}) \quad \text{where} \quad \mathbf{a_1} \in \mathbb{R}^{128}$$

4. **Second Fully Connected Layer (fc2)**:

$$\mathbf{z_2} = \mathbf{W_2}\mathbf{a_1} + \mathbf{b_2} \quad \text{where} \quad \mathbf{W_2} \in \mathbb{R}^{64\times128}, \quad \mathbf{b_2} \in \mathbb{R}^{64}$$

5. **ReLU Activation (after fc2)**:

$$\mathbf{a_2} = \text{ReLU}(\mathbf{z_2}) = \max(0, \mathbf{z_2}) \quad \text{where} \quad \mathbf{a_2} \in \mathbb{R}^{64}$$

6. **Third Fully Connected Layer (fc3)**:

$$\mathbf{z_3} = \mathbf{W_3}\mathbf{a_2} + \mathbf{b_3} \quad \text{where} \quad \mathbf{W_3} \in \mathbb{R}^{10\times64}, \quad \mathbf{b_3} \in \mathbb{R}^{10}$$

7. **Output**:

$$\mathbf{y} = \mathbf{z_3} \quad \text{where} \quad \mathbf{y} \in \mathbb{R}^{10}$$

8. **Network Output**: The output of the network can be written as:

$$\mathbf{y} = \mathbf{W_3} \cdot \text{ReLU}(\mathbf{W_2} \cdot \text{ReLU}(\mathbf{W_1} \cdot \mathbf{x}_{\text{flat}} + \mathbf{b_1}) + \mathbf{b_2}) + \mathbf{b_3}$$

9. The final output can be extracted by applying the **Softmax** function on the y.

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

5.2 Back Propagation

The backward pass involves calculating the gradients of the loss with respect to the weights, biases, and intermediate activations in the network using backpropagation. For the Cross-Entropy (CE) Loss function, the gradients are derived as follows:

Notation

- L: Loss function (Cross-Entropy Loss)
- y_{true}: Ground truth labels (one-hot encoded)
- y_{pred}: Predicted output (softmax probabilities)
- W_i, b_i: Weights and biases of the i-th layer
- a_i, z_i: Activations and pre-activation values for the i-th layer

Loss Derivative

The gradient of the Cross-Entropy Loss with respect to the output logits of the final layer:

$$\frac{\partial L}{\partial z_3} = y_{\text{pred}} - y_{\text{true}}$$

Gradients for Fully Connected Layers

Third Fully Connected Layer ($fc3$):

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial z_3} \cdot a_2^T$$
$$\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial z_3}$$
$$\frac{\partial L}{\partial a_2} = W_3^T \cdot \frac{\partial L}{\partial z_3}$$

Second Fully Connected Layer ($fc2$):

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial \text{ReLU}(z_2)}{\partial z_2}$$
$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \cdot a_1^T$$
$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2}$$
$$\frac{\partial L}{\partial a_1} = W_2^T \cdot \frac{\partial L}{\partial z_2}$$

First Fully Connected Layer ($fc1$):

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \frac{\partial \text{ReLU}(z_1)}{\partial z_1}$$
$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \cdot x_{\text{flat}}^T$$
$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1}$$

Backpropagation through Activation Functions

ReLU Activation: The derivative of the ReLU activation function is given by:

$$\frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Softmax with Cross-Entropy Loss: For the softmax function combined with Cross-Entropy Loss, the gradient simplifies to:

$$\frac{\partial L}{\partial z_3} = y_{\text{pred}} - y_{\text{true}}$$

Summary of Backpropagation Flow

The gradients for each layer are computed in sequence:

1. Compute $\frac{\partial L}{\partial z_3}$ from the Cross-Entropy Loss.
2. Propagate through $fc3$ to compute $\frac{\partial L}{\partial W_3}$, $\frac{\partial L}{\partial b_3}$, and $\frac{\partial L}{\partial a_2}$.
3. Propagate through $fc2$ to compute $\frac{\partial L}{\partial W_2}$, $\frac{\partial L}{\partial b_2}$, and $\frac{\partial L}{\partial a_1}$.
4. Propagate through $fc1$ to compute $\frac{\partial L}{\partial W_1}$ and $\frac{\partial L}{\partial b_1}$.

The computed gradients are then used to update the weights and biases during the optimization step.

6 Model Parameters

Below is a reconstruction of the simplistic model for MNIST handwritten digit classification.

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

6.1 Input Layer

Accepts the 784-element vector from the flattened image.

The input source can be understood to be of the following structure.

$$fixed{-}point[batch_size, num_channels, height, width]$$

Wherein, the following values are known at the time of designing.

- $num_channels(1)$ for gray scale image.
- $height(28)$ Specific input size for the MNIST data set.
- $width(28)$ Specific input size for the MNIST data set.

6.2 Flatten Layer

The **Flatten Layer** reshapes the input tensor into a 1D vector of size 1×784. This prepares the data for the dense (Gemm) layers by collapsing the spatial dimensions.

- **Input Shape:** fixed-point$[batch_size, 1, 28, 28]$
- **Output Shape:** fixed-point$[batch_size, 784]$

6.3 Fully Connected Layer 1 (Gemm Layer 1)

This layer performs a matrix multiplication followed by an addition of a bias vector.

- **Weight Matrix** B**:** 128×784
- **Bias Vector** C**:** 128
- **Activation:** None (pre-ReLU stage)

Computation:

$$\mathbf{z}_1 = \mathbf{x} \cdot \mathbf{B}_1^T + \mathbf{C}_1$$

- **Input Shape:** fixed-point$[batch_size, 784]$
- **Output Shape:** fixed-point$[batch_size, 128]$

6.4 Activation Layer 1 (ReLU)

This applies the **Rectified Linear Unit (ReLU)** activation function element-wise to the output of Gemm Layer 1.

ReLU Function:

$$\mathrm{ReLU}(x) = \max(0, x)$$

- **Input Shape:** fixed-point$[batch_size, 128]$
- **Output Shape:** fixed-point$[batch_size, 128]$

6.5 Fully Connected Layer 2 (Gemm Layer 2)

This second dense layer performs another matrix multiplication and bias addition.

- **Weight Matrix** B**:** 64×128
- **Bias Vector** C**:** 64
- **Activation:** None (pre-ReLU stage)

Computation:

$$\mathbf{z}_2 = \mathbf{h}_1 \cdot \mathbf{B}_2^T + \mathbf{C}_2$$

- **Input Shape:** fixed-point$[batch_size, 128]$
- **Output Shape:** fixed-point$[batch_size, 64]$

6.6 Activation Layer 2 (ReLU)

The second ReLU activation is applied element-wise to the output of Gemm Layer 2.

- **Input Shape:** fixed-point$[batch_size, 64]$
- **Output Shape:** fixed-point$[batch_size, 64]$

6.7 Fully Connected Layer 3 (Gemm Layer 3)

The final dense layer reduces the dimensionality to match the 10 output classes.

- **Weight Matrix** B**:** 10×64

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

- **Bias Vector C:** 10
- **Activation:** None

Computation:

$$\mathbf{z}_3 = \mathbf{h}_2 \cdot \mathbf{B}_3^T + \mathbf{C}_3$$

- **Input Shape:** fixed-point$[batch_size, 64]$
- **Output Shape:** fixed-point$[batch_size, 10]$

6.8 Output Layer (Softmax)

The final output is passed through the **Softmax function** to convert logits into probabilities across the 10 classes.

Softmax Function:

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{10} \exp(z_j)}$$

- **Input Shape:** fixed-point$[batch_size, 10]$
- **Output Shape:** fixed-point$[batch_size, 10]$

Each value represents the predicted probability of the corresponding class. The class with the highest probability is selected as the final output.

References

[1] Sharan Chetlur et al. cudnn: Efficient primitives for deep learning. In *arXiv preprint arXiv:1410.0759*, 2014.

[2] Jacob Fosso-Tande. Applications of taylor series. 08 2013.

[3] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[5] Alex Krizhevsky. Learning multiple layers of features from tiny images. *Technical Report*, 2012.

[6] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[7] Yuanqing Liang et al. Fpga-based image recognition framework. *Journal of Parallel and Distributed Computing*, 120:15–25, 2018.

[8] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[9] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.

[10] Yaman Umuroglu et al. Finn: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 65–74, 2017.

[11] Stylianos I Venieris and Christos-Savvas Bouganis. f-cnnx: A toolflow for mapping cnns on fpgas. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 11(3):1–24, 2018.

[12] Chen Zhang, Peng Li, Guangyu Sun, Yuxin Guan, Bingjun Xiao, and Jason Cong. Optimizing fpga-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 161–170, 2015.

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

A Exemplified Training in Python (Simplified)

The following Python code demonstrates the training process:

```
import numpy as np
from tensorflow.keras.datasets import mnist

# Set precision to bfloat16 (emulated)
def to_bfloat16(x):
    x = x.astype(np.float32)  # Convert to float32
    view = x.view(np.uint32)  # View as 32-bit integers
    view &= 0xFFFF0000        # Truncate lower 16 bits
    return view.view(np.float32)  # Back to float32 view

# Load MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# Normalize the dataset
train_images = to_bfloat16(train_images / 255.0)
test_images = to_bfloat16(test_images / 255.0)
train_labels = np.eye(10)[train_labels].astype(np.float32)  # One-hot encoding
test_labels = np.eye(10)[test_labels].astype(np.float32)

# Flatten images
train_images = train_images.reshape(-1, 28 * 28)
test_images = test_images.reshape(-1, 28 * 28)

# Neural network parameters
input_size = 28 * 28
hidden_size1 = 128
hidden_size2 = 64
output_size = 10
learning_rate = 0.001
num_epochs = 50
batch_size = 64//2

# Initialize weights and biases with bfloat16 precision
np.random.seed(42)
weights1 = to_bfloat16(np.random.randn(input_size, hidden_size1) * np.sqrt(2.0 / (input_siz
weights2 = to_bfloat16(np.random.randn(hidden_size1, hidden_size2) * np.sqrt(2.0 / (hidden
weights3 = to_bfloat16(np.random.randn(hidden_size2, output_size) * np.sqrt(2.0 / (hidden_
bias1 = to_bfloat16(np.zeros((1, hidden_size1)))
bias2 = to_bfloat16(np.zeros((1, hidden_size2)))
bias3 = to_bfloat16(np.zeros((1, output_size)))

# Activation functions
```

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

```
def relu(x):
    return np.maximum(0, x)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

def relu_derivative(x):
    return (x > 0).astype(np.float32)

# Gradient clipping function
def clip_gradients(grad, threshold=1.0):
    return np.clip(grad, -threshold, threshold)

# Training loop
for epoch in range(num_epochs):
    # Shuffle data
    indices = np.arange(len(train_images))
    np.random.shuffle(indices)
    train_images = train_images[indices]
    train_labels = train_labels[indices]

    for i in range(0, len(train_images), batch_size):
        # Get batch
        x_batch = train_images[i:i+batch_size]
        y_batch = train_labels[i:i+batch_size]

        # Forward pass
        z1 = to_bfloat16(np.dot(x_batch, weights1) + bias1)
        a1 = relu(z1)
        z2 = to_bfloat16(np.dot(a1, weights2) + bias2)
        a2 = relu(z2)
        z3 = to_bfloat16(np.dot(a2, weights3) + bias3)
        predictions = softmax(z3)

        # Compute loss (categorical cross-entropy)
        loss = -np.mean(np.sum(y_batch * np.log(predictions + 1e-7), axis=1))

        # Backward pass
        dz3 = predictions - y_batch
        dw3 = to_bfloat16(np.dot(a2.T, dz3) / batch_size)
        db3 = to_bfloat16(np.sum(dz3, axis=0, keepdims=True) / batch_size)

        dz2 = np.dot(dz3, weights3.T) * relu_derivative(a2)
        dw2 = to_bfloat16(np.dot(a1.T, dz2) / batch_size)
        db2 = to_bfloat16(np.sum(dz2, axis=0, keepdims=True) / batch_size)
```

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY

```
        dz1 = np.dot(dz2, weights2.T) * relu_derivative(a1)
        dw1 = to_bfloat16(np.dot(x_batch.T, dz1) / batch_size)
        db1 = to_bfloat16(np.sum(dz1, axis=0, keepdims=True) / batch_size)

        # Clip gradients
        dw3 = clip_gradients(dw3)
        db3 = clip_gradients(db3)
        dw2 = clip_gradients(dw2)
        db2 = clip_gradients(db2)
        dw1 = clip_gradients(dw1)
        db1 = clip_gradients(db1)

        # Update weights and biases
        weights1 -= learning_rate * dw1
        bias1 -= learning_rate * db1
        weights2 -= learning_rate * dw2
        bias2 -= learning_rate * db2
        weights3 -= learning_rate * dw3
        bias3 -= learning_rate * db3

    print(f"Epoch {epoch + 1}/{num_epochs}, Loss: {loss:.4f}")

# Evaluate on test data
z1 = to_bfloat16(np.dot(test_images, weights1) + bias1)
a1 = relu(z1)
z2 = to_bfloat16(np.dot(a1, weights2) + bias2)
a2 = relu(z2)
z3 = to_bfloat16(np.dot(a2, weights3) + bias3)
predictions = softmax(z3)

accuracy = np.mean(np.argmax(predictions, axis=1) == np.argmax(test_labels, axis=1))
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

AGNISYS
SYSTEM DEVELOPMENT WITH CERTAINTY